Chapter 1

Reinforcement Learning

This chapter introduces the fundamental concepts of reinforcement learning, providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the standard references in reinforcement learning, in particular the works of Sutton and Barto [**?**]. Its structure is as follows:

- Historical developments: an overview of the key ideas and milestones that led to the emergence of reinforcement learning as a unified field.
- Markov Decision Processes (MDPs): a formalization of how reinforcement learning problems are structured and modeled.
- Actor and critic roles: a presentation of the respective roles of the policy (actor) and value estimation (critic).
- Actor–critic learning: an explanation of how the actor and critic are jointly trained through interaction with the environment.
- Policy evaluation: a discussion of how learned policies are assessed and compared.

1.1 Historical Developments

Reinforcement learning is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. Its development can be understood as a chronological evolution from behavioral theories of learning, to mathematical optimal control, and finally to data-driven methods.

In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

1.1.1 Behavioral foundations (1900–1950)

The earliest roots of reinforcement learning can be traced back to psychological studies of learning behavior. The foundational idea is the principle that actions followed by satisfying outcomes become more likely to be repeated, while those followed by negative outcomes tend to be avoided. This establishes a basic mechanism of learning through interaction with consequences [Thorndike, 1911].

This perspective was later developed within behaviorism. In this school of psychology, reinforcement and punishment are used to gradually shape behavior through repeated interaction with the environment [Skinner, 2019]. The term ”reinforcement” in the context of learning comes from this school of thought. This perspective in behaviorism influenced early thinking in computing and artificial intelligence. It led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a “pleasure–pain system” [Turing, 2004].

Although these studies did not involve formal mathematical models, they introduced key ideas that later became central to reinforcement learning, including trial-and-error learning and adaptation driven by environmental feedback.

1.1.2 Sequential decision-making (1950–1970)

A second major foundation emerged from the field of optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. The problem is to choose actions over time in order to maximize long-term performance rather than immediate reward. A breakthrough was introduced through dynamic programming [Bellman, 1957], which provided a recursive decomposition of sequential decision problems via the principle of optimality.

This framework led to the formalization of Markov Decision Processes (MDPs), which provide a mathematical model for sequential decision-making under uncertainty [Howard, 1960]. MDPs explicitly combine states, actions, rewards, and environment dynamics into a unified stochastic process, where the goal is to maximize expected cumulative reward over time.

Classical dynamic programming methods rely on the assumption that the environment dynamics are fully known. This requirement implies that the full state space and all transition relationships must be explicitly represented. This quickly becomes computationally intractable in realistic problems.

1.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference (TD) learning methods were introduced [Sutton, 1988]. TD learning combines ideas from Monte Carlo methods [**?**] and dynamic programming by updating predictions based on other learned estimates, rather than waiting for final outcomes.

This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations and forms a key building block of reinforcement learning algorithms.

Building on these ideas, Q-learning was introduced, showing that optimal action-value functions can be learned directly from interaction with the environment without requiring a model of its dynamics [Watkins, 1989, Watkins and Dayan, 1992]. This period culminated in the formalization of reinforcement learning as a coherent field, through the publication of a foundational reference work [Sutton and Barto, 2018].

1.1.4 Consolidation of reinforcement learning (2000–2013)

A key limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDPs, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced, allowing these quantities to be represented using parametric models such as linear approximators or perceptron-like architectures [Barto et al., 1983, Tsitsiklis and Van Roy, 1996]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space.

The introduction of Deep Q-Networks (DQN), which demonstrated that deep neural networks could be used to approximate value functions at scale while maintaining stability. By combining Q-learning with deep neuralnetwor architectures, DQN successfully extended value-based reinforcement learning to high-dimensional input spaces such as raw pixels in Atari games [Mnih et al., 2013].

1.1.5 Deep reinforcement learning (2013–2026)

Bibliography

[Barto et al., 1983] Barto, A. G., Sutton, R. S., and Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics, 13(5):835–846.

[Bellman, 1957] Bellman, R. E. (1957). Dynamic programming.

[Howard, 1960] Howard, R. A. (1960). Dynamic programming and markov processes. MIT Press.

[Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

[Skinner, 2019] Skinner, B. F. (2019). The behavior of organisms: An experimental analysis. BF Skinner Foundation.

[Sutton, 1988] Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine Learning, 3:9–44.

[Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press, 2nd edition.

[Thorndike, 1911] Thorndike, E. L. (1911). Animal intelligence: Experimental studies. The Psychological Review Monograph Supplement, 8(4).

[Tsitsiklis and Van Roy, 1996] Tsitsiklis, J. and Van Roy, B. (1996). Analysis of temporal-diffference learning with function approximation. Advances in neural information processing systems, 9.

[Turing, 2004] Turing, A. (2004). Intelligent machinery (1948). B. Jack Copeland, page 395.

[Watkins, 1989] Watkins, C. J. C. H. (1989). Learning from delayed rewards. PhD Thesis, University of Cambridge.

[Watkins and Dayan, 1992] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. Machine Learning, 8:279–292.